
SQL Queries

1. Data Query Language (DQL) – `SELECT`

DQL is used to fetch data from a database. The primary command is:

Basic `SELECT` Examples

- **Select All Columns:**
- `SELECT * FROM employees;`

Retrieves every column from the `employees` table.

- **Select Specific Columns:**
- `SELECT first_name, last_name FROM employees;`

Fetches only `first_name` and `last_name`.

SQL `WHERE` Clause — Complete Explanation with Examples

The `WHERE` clause in SQL is used to **filter records** from a table based on a specified condition. It returns only those rows that meet the condition.

Basic Syntax

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Common Use Cases of `WHERE`

1. Equal to (=)

```
SELECT * FROM employees  
WHERE department = 'Sales';
```

Fetches all employees in the Sales department.

2. Not Equal To (`!=` or `<>`)

```
SELECT * FROM employees
WHERE department <> 'HR';
```

Returns all employees who are not in HR.

3. Greater Than / Less Than

```
SELECT * FROM products
WHERE price > 100;
```

Gets products with a price over 100.

```
SELECT * FROM employees
WHERE age < 30;
```

Employees younger than 30.

4. Using AND, OR, NOT

```
SELECT * FROM employees
WHERE department = 'IT' AND salary > 60000;
```

IT employees earning more than 60,000.

```
SELECT * FROM employees
WHERE department = 'Finance' OR department = 'HR';
```

Employees in either Finance or HR.

```
SELECT * FROM employees
WHERE NOT (department = 'IT');
```

All employees not in the IT department.

5. Using BETWEEN

```
SELECT * FROM products
WHERE price BETWEEN 100 AND 500;
```

Products priced between 100 and 500.

6. Using IN

```
SELECT * FROM employees
WHERE department IN ('HR', 'Finance', 'Marketing');
```

Employees from multiple specified departments.

7. Using LIKE

```
SELECT * FROM employees
WHERE first_name LIKE 'A%';
```

First names that start with 'A'.

8. Using IS NULL or IS NOT NULL

```
SELECT * FROM employees
WHERE manager_id IS NULL;
```

Employees without a manager.

```
SELECT * FROM employees
WHERE manager_id IS NOT NULL;
```

Employees who report to someone.

2. Relational Operators

Used to compare values in SQL.

Operator	Description	Example
=	Equal to	salary = 50000
<> or !=	Not equal to	department_id <> 10
>	Greater than	price > 100
<	Less than	age < 30
>=	Greater than or equal to	salary >= 60000
<=	Less than or equal to	quantity <= 100

Example:

```
SELECT * FROM employees
WHERE salary >= 50000 AND department_id != 3;
```

3. Logical Operators

Combine multiple conditions.

Operator	Description
AND	True if both conditions are true
OR	True if any condition is true
NOT	Negates the condition

Examples:

- **AND:**
 - SELECT * FROM products
 - WHERE category = 'Electronics' AND price > 1000;
- **OR:**
 - SELECT * FROM employees
 - WHERE department = 'HR' OR department = 'Finance';

- **NOT:**
 - `SELECT * FROM employees`
 - `WHERE NOT (department = 'IT');`
-

4. **LIKE** Operator (Pattern Matching)

Used to search for a specific pattern in a column.

Pattern	Description	Example Match
'A%'	Starts with A	Adam, Alice
'%son'	Ends with 'son'	Jackson
'%man%'	Contains 'man'	Human, Manual
'M_n'	'M' + any 1 char + 'n'	Man, Men

Example:

```
SELECT * FROM employees
WHERE first_name LIKE 'A%';
```

5. Concatenation (Joining Strings)

Combine multiple strings into one.

MySQL:

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM employees;
```

PostgreSQL / Oracle:

```
SELECT first_name || ' ' || last_name AS full_name
FROM employees;
```

6. Aggregate Functions

Perform calculations across a set of rows.

COUNT () – Total number of rows

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

SUM () – Total sum of a numeric column

```
SELECT SUM(salary) AS total_salary
FROM employees;
```

AVG () – Average of a column

```
SELECT AVG(salary) AS avg_salary
FROM employees;
```

MIN () – Minimum value

```
SELECT MIN(salary) AS min_salary
FROM employees;
```

MAX () – Maximum value

```
SELECT MAX(salary) AS max_salary
FROM employees;
```

7. Using GROUP BY

Groups rows and allows aggregate functions per group.

Example:

```
SELECT department_id, AVG(salary) AS avg_dept_salary
FROM employees
GROUP BY department_id;
```

8. HAVING Clause

Filters **groups** (like `WHERE` but after grouping).

Example:

```
SELECT department_id, COUNT(*) AS employee_count
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

9. ORDER BY Clause

Used to **sort results** in ascending (`ASC`) or descending (`DESC`) order.

Sort by One Column

```
SELECT * FROM employees
ORDER BY salary DESC;
```

Employees listed from highest to lowest salary.

Sort by Multiple Columns

```
SELECT * FROM employees
```

```
ORDER BY department_id ASC, salary DESC;
```

Sorts first by department ID (ascending), then salary (descending) within each department.

10. Combined Example Query

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name, salary
```

```
FROM employees
```

```
WHERE department = 'IT'
```

```
    AND salary > 60000
```

```
    AND first_name LIKE 'J%'
```

```
ORDER BY salary DESC;
```

11. Aliases (AS)

An alias gives a **temporary name** to a table or column—useful for readability or when using functions.

Column Alias

```
SELECT first_name AS fname, last_name AS lname
```

```
FROM employees;
```

Temporarily renames columns for easier output formatting.

Table Alias

```
SELECT e.first_name, d.department_name
```

```
FROM employees AS e
```

```
JOIN departments AS d ON e.department_id = d.id;
```

Shortens table names for convenience, especially in complex queries.

12. SQL JOIN – Combining Data from Multiple Tables

JOIN combines rows from two or more tables based on a related column.

Types of Joins

Join Type	Description
INNER JOIN	Returns only matching rows from both tables
LEFT JOIN	Returns all rows from the left table, and matched rows from the right
RIGHT JOIN	Returns all rows from the right table, and matched rows from the left

Join Type	Description
FULL JOIN	Returns rows when there is a match in either table (supported in some DBs)
CROSS JOIN	Returns Cartesian product (all combinations)

INNER JOIN Example

```
SELECT e.first_name, d.department_name
FROM employees e
INNER JOIN departments d ON e.department_id = d.id;
```

Returns employees with valid department matches.

LEFT JOIN Example

```
SELECT e.first_name, d.department_name
FROM employees e
LEFT JOIN departments d ON e.department_id = d.id;
```

Returns all employees, including those who don't belong to any department (NULL values for unmatched).

RIGHT JOIN Example

```
SELECT e.first_name, d.department_name
FROM employees e
RIGHT JOIN departments d ON e.department_id = d.id;
```

Returns all departments, even if no employees are assigned to them.

FULL OUTER JOIN Example (if supported)

```
SELECT e.first_name, d.department_name
FROM employees e
FULL OUTER JOIN departments d ON e.department_id = d.id;
```

Returns all records from both tables, with NULLs where there's no match.

CROSS JOIN Example

```
SELECT a.product_name, b.customer_name
FROM products a
CROSS JOIN customers b;
```

Every product matched with every customer — used for analysis, testing, or generating combos.
